

Программная реализация сетевого сервера

Создано системой Doxygen 1.9.4

1	Алфавитный указатель классов	1
1.1	Классы	1
2	Список файлов	3
2.1	Файлы	3
3	Классы	5
3.1	Класс Authenticator	5
3.1.1	Подробное описание	5
3.1.2	Методы	6
3.1.2.1	authenticate()	6
3.1.2.2	computeSHA1()	6
3.1.2.3	generateSalt()	7
3.2	Класс ClientBase	7
3.2.1	Подробное описание	8
3.2.2	Конструктор(ы)	8
3.2.2.1	ClientBase()	8
3.2.3	Методы	8
3.2.3.1	getPassword()	9
3.2.3.2	load()	9
3.2.3.3	userExists()	9
3.2.4	Данные класса	10
3.2.4.1	clients	10
3.2.4.2	filename	10
3.3	Класс ClientHandler	10
3.3.1	Подробное описание	11
3.3.2	Конструктор(ы)	12
3.3.2.1	ClientHandler()	12
3.3.3	Методы	12
3.3.3.1	authenticateClient()	12
3.3.3.2	handle()	13
3.3.3.3	receiveData()	13
3.3.3.4	receiveString()	14
3.3.3.5	receiveVectorsAndProcess()	14
3.3.3.6	sendData()	15
3.3.3.7	sendMessage()	16
3.3.3.8	sendSalt()	16
3.3.4	Данные класса	17
3.3.4.1	clientAddr	17
3.3.4.2	clientBase	17
3.3.4.3	clientSocket	17
3.3.4.4	logger	18
3.4	Класс CommandLineParser	18
3.4.1	Подробное описание	19

3.4.2	Конструктор(ы)	19
3.4.2.1	CommandLineParser()	19
3.4.3	Методы	19
3.4.3.1	getConfig()	20
3.4.3.2	getHelp()	20
3.4.3.3	parse()	20
3.4.3.4	validateRequiredParams()	21
3.4.4	Данные класса	21
3.4.4.1	config	21
3.4.4.2	helpText	21
3.5	Класс Logger	21
3.5.1	Подробное описание	22
3.5.2	Конструктор(ы)	23
3.5.2.1	Logger()	23
3.5.2.2	~Logger()	23
3.5.3	Методы	23
3.5.3.1	error()	23
3.5.3.2	getCurrentTime()	24
3.5.3.3	info()	24
3.5.3.4	levelToString()	24
3.5.3.5	log()	25
3.5.3.6	warning()	25
3.5.4	Данные класса	25
3.5.4.1	logFile	25
3.5.4.2	logMutex	26
3.6	Класс Server	26
3.6.1	Подробное описание	27
3.6.2	Конструктор(ы)	27
3.6.2.1	Server()	27
3.6.2.2	~Server()	27
3.6.3	Методы	28
3.6.3.1	cleanup()	28
3.6.3.2	initialize()	28
3.6.3.3	run()	28
3.6.3.4	stop()	28
3.6.4	Данные класса	29
3.6.4.1	clientBase	29
3.6.4.2	clientBaseFile	29
3.6.4.3	logFile	29
3.6.4.4	logger	29
3.6.4.5	port	29
3.6.4.6	running	30
3.6.4.7	serverSocket	30

3.7 Структура ServerConfig	30
3.7.1 Подробное описание	30
3.7.2 Данные класса	30
3.7.2.1 clientBaseFile	31
3.7.2.2 logFile	31
3.7.2.3 port	31
3.8 Класс VectorCalculator	31
3.8.1 Подробное описание	32
3.8.2 Методы	32
3.8.2.1 computeProduct()	32
4 Файлы	35
4.1 Файл Authenticator.cpp	35
4.2 Authenticator.cpp	35
4.3 Файл Authenticator.h	36
4.4 Authenticator.h	37
4.5 Файл ClientBase.cpp	38
4.6 ClientBase.cpp	38
4.7 Файл ClientBase.h	39
4.8 ClientBase.h	40
4.9 Файл ClientHandler.cpp	40
4.9.1 Макросы	40
4.9.1.1 be64toh	40
4.9.1.2 htobe64	41
4.10 ClientHandler.cpp	41
4.11 Файл ClientHandler.h	44
4.12 ClientHandler.h	45
4.13 Файл CommandLineParser.cpp	46
4.14 CommandLineParser.cpp	46
4.15 Файл CommandLineParser.h	48
4.16 CommandLineParser.h	49
4.17 Файл Logger.cpp	49
4.18 Logger.cpp	49
4.19 Файл Logger.h	50
4.19.1 Перечисления	51
4.19.1.1 LogLevel	51
4.20 Logger.h	52
4.21 Файл main.cpp	52
4.21.1 Функции	53
4.21.1.1 main()	53
4.21.1.2 signalHandler()	54
4.21.2 Переменные	54
4.21.2.1 server	54

4.22 main.cpp	54
4.23 Файл Server.cpp	55
4.24 Server.cpp	55
4.25 Файл Server.h	56
4.26 Server.h	57
4.27 Файл VectorCalculator.cpp	58
4.28 VectorCalculator.cpp	58
4.29 Файл VectorCalculator.h	59
4.30 VectorCalculator.h	60
Предметный указатель	61

Глава 1

Алфавитный указатель классов

1.1 Классы

Классы с их кратким описанием.

Authenticator	Класс для аутентификации клиентов с использованием Crypto++	5
ClientBase	Класс для работы с базой данных клиентов	7
ClientHandler	Класс для обработки клиентских подключений	10
CommandLineParser	Класс для разбора командной строки с использованием getopt/getopt_long	18
Logger	Класс для логирования событий сервера	21
Server	Основной класс сервера	26
ServerConfig	Структура для хранения конфигурации сервера	30
VectorCalculator	Класс для вычисления произведения элементов вектора	31

Глава 2

Список файлов

2.1 Файлы

Полный список файлов.

Authenticator.cpp	35
Authenticator.h	36
ClientBase.cpp	38
ClientBase.h	39
ClientHandler.cpp	40
ClientHandler.h	44
CommandLineParser.cpp	46
CommandLineParser.h	48
Logger.cpp	49
Logger.h	50
main.cpp	52
Server.cpp	55
Server.h	56
VectorCalculator.cpp	58
VectorCalculator.h	59

Глава 3

Классы

3.1 Класс Authenticator

Класс для аутентификации клиентов с использованием Crypto++.

```
#include <Authenticator.h>
```

Открытые статические члены

- static std::string [generateSalt](#) ()
Генерация случайной соли с помощью Crypto++.
- static std::string [computeSHA1](#) (const std::string &data)
Вычисление SHA1 хеша от данных с помощью Crypto++.
- static bool [authenticate](#) (const std::string &password, const std::string &salt, const std::string &clientHash)
Аутентификация клиента

3.1.1 Подробное описание

Класс для аутентификации клиентов с использованием Crypto++.

Автор

Гришин Николай

Версия

1.0

Дата

24.11.2025

Авторство

ИБСТ ПГУ

См. определение в файле [Authenticator.h](#) строка 18

3.1.2 Методы

3.1.2.1 authenticate()

```
bool Authenticator::authenticate (
    const std::string & password,
    const std::string & salt,
    const std::string & clientHash ) [static]
```

Аутентификация клиента

Аргументы

password	Пароль пользователя
salt	Соль для аутентификации
clientHash	Хеш от клиента

Возвращает

true если аутентификация успешна, false в противном случае

См. определение в файле [Authenticator.cpp](#) строка 73

3.1.2.2 computeSHA1()

```
std::string Authenticator::computeSHA1 (
    const std::string & data ) [static]
```

Вычисление SHA1 хеша от данных с помощью Crypto++.

Аргументы

data	Данные для хеширования
------	------------------------

Возвращает

Хеш в виде строки шестнадцатеричных цифр

См. определение в файле [Authenticator.cpp](#) строка 51

3.1.2.3 generateSalt()

```
std::string Authenticator::generateSalt ( ) [static]
```

Генерация случайной соли с помощью Crypto++.

Возвращает

Соль в виде строки из 16 шестнадцатеричных цифр

См. определение в файле [Authenticator.cpp](#) строка 18

Объявления и описания членов классов находятся в файлах:

- [Authenticator.h](#)
- [Authenticator.cpp](#)

3.2 Класс ClientBase

Класс для работы с базой данных клиентов

```
#include <ClientBase.h>
```

Открытые члены

- [ClientBase](#) (const std::string &filename)
Конструктор класса [ClientBase](#).
- bool [load](#) ()
Загрузка базы данных клиентов из файла
- bool [userExists](#) (const std::string &login) const
Проверка существования пользователя в базе
- std::string [getPassword](#) (const std::string &login) const
Получение пароля пользователя

Закрытые данные

- std::string [filename](#)
Имя файла с базой данных
- std::unordered_map< std::string, std::string > [clients](#)
Контейнер для хранения клиентов

3.2.1 Подробное описание

Класс для работы с базой данных клиентов

Автор

Гришин Николай

Версия

1.0

Дата

24.11.2025

Авторство

ИБСТ ПГУ

См. определение в файле [ClientBase.h](#) строка 14

3.2.2 Конструктор(ы)

3.2.2.1 ClientBase()

```
ClientBase::ClientBase (  
    const std::string & filename )
```

Конструктор класса [ClientBase](#).

Аргументы

filename	Имя файла с базой данных клиентов
----------	-----------------------------------

См. определение в файле [ClientBase.cpp](#) строка 10

3.2.3 Методы

3.2.3.1 getPassword()

```
std::string ClientBase::getPassword (
    const std::string & login ) const
```

Получение пароля пользователя

Аргументы

login	Логин пользователя
-------	--------------------

Возвращает

Пароль пользователя или пустую строку если пользователь не найден

См. определение в файле [ClientBase.cpp](#) строка 50

3.2.3.2 load()

```
bool ClientBase::load ( )
```

Загрузка базы данных клиентов из файла

Возвращает

true если загрузка успешна, false в противном случае

См. определение в файле [ClientBase.cpp](#) строка 16

3.2.3.3 userExists()

```
bool ClientBase::userExists (
    const std::string & login ) const
```

Проверка существования пользователя в базе

Аргументы

login	Логин пользователя
-------	--------------------

Возвращает

true если пользователь существует, false в противном случае

См. определение в файле [ClientBase.cpp](#) строка 41

3.2.4 Данные класса

3.2.4.1 clients

```
std::unordered_map<std::string, std::string> ClientBase::clients [private]
```

Контейнер для хранения клиентов

См. определение в файле [ClientBase.h](#) строка 44

3.2.4.2 filename

```
std::string ClientBase::filename [private]
```

Имя файла с базой данных

См. определение в файле [ClientBase.h](#) строка 43

Объявления и описания членов классов находятся в файлах:

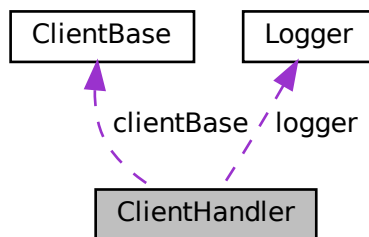
- [ClientBase.h](#)
- [ClientBase.cpp](#)

3.3 Класс ClientHandler

Класс для обработки клиентских подключений

```
#include <ClientHandler.h>
```

Граф связей класса ClientHandler:



Открытые члены

- [ClientHandler](#) (int [clientSocket](#), const sockaddr_in &[clientAddr](#), [ClientBase](#) &[clientBase](#), [Logger](#) &[logger](#))
Конструктор класса [ClientHandler](#).
- void [handle](#) ()
Основной метод обработки клиента

Закрытые члены

- bool [authenticateClient](#) ()
Аутентификация клиента
- bool [receiveVectorsAndProcess](#) ()
Прием и обработка векторов от клиента
- bool [sendMessage](#) (const std::string &message)
Отправка текстового сообщения клиенту
- bool [receiveData](#) (void *buffer, size_t size)
Прием данных от клиента
- bool [sendData](#) (const void *data, size_t size)
Отправка данных клиенту
- bool [receiveString](#) (std::string &result, size_t maxSize=256)
Прием строки от клиента
- bool [sendSalt](#) (const std::string &salt)
Отправка соли клиенту

Закрытые данные

- int [clientSocket](#)
Сокет клиента
- sockaddr_in [clientAddr](#)
Адрес клиента
- [ClientBase](#) & [clientBase](#)
Ссылка на базу клиентов
- [Logger](#) & [logger](#)
Ссылка на логгер

3.3.1 Подробное описание

Класс для обработки клиентских подключений

Автор

Гришин Николай

Версия

1.0

Дата

24.11.2025

Авторство

ИБСТ ПГУ

См. определение в файле [ClientHandler.h](#) строка 21

3.3.2 Конструктор(ы)

3.3.2.1 ClientHandler()

```
ClientHandler::ClientHandler (
    int clientSocket,
    const sockaddr_in & clientAddr,
    ClientBase & clientBase,
    Logger & logger )
```

Конструктор класса [ClientHandler](#).

Аргументы

clientSocket	Сокет клиента
clientAddr	Адрес клиента
clientBase	База данных клиентов
logger	Логгер для записи событий

См. определение в файле [ClientHandler.cpp](#) строка [25](#)

3.3.3 Методы

3.3.3.1 authenticateClient()

```
bool ClientHandler::authenticateClient ( ) [private]
```

Аутентификация клиента

Аутентификация клиента по протоколу

Возвращает

true если аутентификация успешна, false в противном случае
true если аутентификация успешна, false в противном случае

Выполняет следующие шаги:

1. Прием логина от клиента
2. Проверка существования пользователя в базе
3. Генерация и отправка соли
4. Прием хеша от клиента
5. Проверка хеша аутентификации

См. определение в файле [ClientHandler.cpp](#) строка [71](#)

3.3.3.2 handle()

```
void ClientHandler::handle ( )
```

Основной метод обработки клиента

Основной метод обработки клиентского подключения

Выполняет аутентификацию клиента и обработку векторов данных

См. определение в файле [ClientHandler.cpp](#) строка 34

3.3.3.3 receiveData()

```
bool ClientHandler::receiveData (
    void * buffer,
    size_t size ) [private]
```

Прием данных от клиента

Прием бинарных данных от клиента

Аргументы

buffer	Буфер для приема данных
size	Размер данных

Возвращает

true если прием успешен, false в противном случае

Аргументы

buffer	Буфер для приема данных
size	Размер данных для приема

Возвращает

true если прием успешен, false в противном случае

Читает точное количество байт без поиска терминатора

См. определение в файле [ClientHandler.cpp](#) строка 226

3.3.3.4 receiveString()

```
bool ClientHandler::receiveString (
    std::string & result,
    size_t maxSize = 256 ) [private]
```

Прием строки от клиента

Аргументы

result	Строка для результата
maxSize	Максимальный размер строки

Возвращает

true если прием успешен, false в противном случае

Аргументы

result	Строка для результата
maxSize	Максимальный размер строки

Возвращает

true если прием успешен, false в противном случае

Использует различные стратегии для приема строки:

- Peek для анализа буфера
- Поиск терминаторов (`\0`, `\r`)
- Резервное чтение побайтно

См. определение в файле [ClientHandler.cpp](#) строка 278

3.3.3.5 receiveVectorsAndProcess()

```
bool ClientHandler::receiveVectorsAndProcess ( ) [private]
```

Прием и обработка векторов от клиента

Прием и обработка векторов данных от клиента

Возвращает

true если обработка успешна, false в противном случае
true если обработка успешна, false в противном случае

Выполняет следующие шаги:

1. Прием количества векторов
2. Для каждого вектора:
 - Прием размера вектора
 - Прием данных вектора
 - Вычисление произведения элементов
 - Отправка результата клиенту

См. определение в файле [ClientHandler.cpp](#) строка 139

3.3.3.6 sendData()

```
bool ClientHandler::sendData (  
    const void * data,  
    size_t size ) [private]
```

Отправка данных клиенту

Отправка бинарных данных клиенту

Аргументы

data	Указатель на данные
size	Размер данных

Возвращает

true если отправка успешна, false в противном случае

Аргументы

data	Указатель на данные для отправки
size	Размер данных для отправки

Возвращает

true если отправка успешна, false в противном случае

Отправляет точное количество байт

См. определение в файле [ClientHandler.cpp](#) строка 252

3.3.3.7 sendMessage()

```
bool ClientHandler::sendMessage (
    const std::string & message ) [private]
```

Отправка текстового сообщения клиенту

Аргументы

message	Сообщение для отправки
---------	------------------------

Возвращает

true если отправка успешна, false в противном случае

Аргументы

message	Сообщение для отправки
---------	------------------------

Возвращает

true если отправка успешна, false в противном случае

Заметки

Отправляет сообщение с нулевым терминатором

См. определение в файле [ClientHandler.cpp](#) строка 214

3.3.3.8 sendSalt()

```
bool ClientHandler::sendSalt (
    const std::string & salt ) [private]
```

Отправка соли клиенту

Аргументы

salt	Соль для отправки
------	-------------------

Возвращает

true если отправка успешна, false в противном случае

Аргументы

salt	Соль для отправки (16 байт)
------	-----------------------------

Возвращает

true если отправка успешна, false в противном случае

Заметки

Отправляет ровно 16 байт без нулевого терминатора

См. определение в файле [ClientHandler.cpp](#) строка 203

3.3.4 Данные класса

3.3.4.1 clientAddr

sockaddr_in ClientHandler::clientAddr [private]

Адрес клиента

См. определение в файле [ClientHandler.h](#) строка 40

3.3.4.2 clientBase

[ClientBase&](#) ClientHandler::clientBase [private]

Ссылка на базу клиентов

См. определение в файле [ClientHandler.h](#) строка 41

3.3.4.3 clientSocket

int ClientHandler::clientSocket [private]

Сокет клиента

См. определение в файле [ClientHandler.h](#) строка 39

3.3.4.4 logger

[Logger](#)& ClientHandler::logger [private]

Ссылка на логгер

См. определение в файле [ClientHandler.h](#) строка 42

Объявления и описания членов классов находятся в файлах:

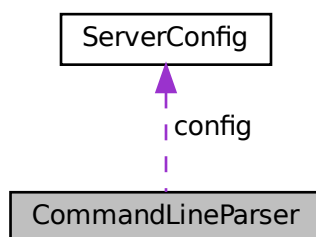
- [ClientHandler.h](#)
- [ClientHandler.cpp](#)

3.4 Класс CommandLineParser

Класс для разбора командной строки с использованием getopt/getopt_long.

```
#include <CommandLineParser.h>
```

Граф связей класса CommandLineParser:



Открытые члены

- [CommandLineParser](#) ()
Конструктор класса [CommandLineParser](#).
- bool [parse](#) (int argc, char *argv[])
Разбор аргументов командной строки с использованием getopt_long.
- [ServerConfig](#) [getConfig](#) () const
Получение конфигурации сервера
- std::string [getHelp](#) () const
Получение текста справки

Закрытые члены

- bool [validateRequiredParams](#) () const
Проверка обязательных параметров

Закрытые данные

- [ServerConfig config](#)
Конфигурация сервера
- `std::string` [helpText](#)
Текст справки

3.4.1 Подробное описание

Класс для разбора командной строки с использованием `getopt/getopt_long`.

Автор

Гришин Николай

Версия

1.0

Дата

24.11.2025

Авторство

ИБСТ ПГУ

См. определение в файле [CommandLineParser.h](#) строка [22](#)

3.4.2 Конструктор(ы)

3.4.2.1 CommandLineParser()

`CommandLineParser::CommandLineParser ( )`

Конструктор класса [CommandLineParser](#).

См. определение в файле [CommandLineParser.cpp](#) строка [14](#)

3.4.3 Методы

3.4.3.1 getConfig()

[ServerConfig](#) CommandLineParser::getConfig () const [inline]

Получение конфигурации сервера

Возвращает

Структура [ServerConfig](#) с настройками

См. определение в файле [CommandLineParser.h](#) строка 41

3.4.3.2 getHelp()

std::string CommandLineParser::getHelp () const

Получение текста справки

Возвращает

Строка с текстом справки

См. определение в файле [CommandLineParser.cpp](#) строка 153

3.4.3.3 parse()

```
bool CommandLineParser::parse (
    int argc,
    char * argv[] )
```

Разбор аргументов командной строки с использованием getopt_long.

Аргументы

argc	Количество аргументов
argv	Массив аргументов

Возвращает

true если разбор успешен, false в противном случае

См. определение в файле [CommandLineParser.cpp](#) строка 58

3.4.3.4 validateRequiredParams()

```
bool CommandLineParser::validateRequiredParams ( ) const [private]
```

Проверка обязательных параметров

Возвращает

true если все обязательные параметры заданы, false в противном случае

См. определение в файле [CommandLineParser.cpp](#) строка 33

3.4.4 Данные класса

3.4.4.1 config

```
ServerConfig CommandLineParser::config [private]
```

Конфигурация сервера

См. определение в файле [CommandLineParser.h](#) строка 50

3.4.4.2 helpText

```
std::string CommandLineParser::helpText [private]
```

Текст справки

См. определение в файле [CommandLineParser.h](#) строка 51

Объявления и описания членов классов находятся в файлах:

- [CommandLineParser.h](#)
- [CommandLineParser.cpp](#)

3.5 Класс Logger

Класс для логирования событий сервера

```
#include <Logger.h>
```

Открытые члены

- [Logger](#) (const std::string &filename)
Конструктор класса [Logger](#).
- [~Logger](#) ()
Деструктор класса [Logger](#).
- void [log](#) ([LogLevel](#) level, const std::string &message)
Основной метод логирования
- void [info](#) (const std::string &message)
Логирование информационного сообщения
- void [warning](#) (const std::string &message)
Логирование предупреждения
- void [error](#) (const std::string &message)
Логирование ошибки

Закрытые члены

- std::string [getCurrentTime](#) ()
Получение текущего времени в формате строки
- std::string [levelToString](#) ([LogLevel](#) level)
Преобразование уровня логирования в строку

Закрытые данные

- std::ofstream [logFile](#)
Файл для записи логов
- std::mutex [logMutex](#)
Мьютекс для синхронизации доступа к файлу

3.5.1 Подробное описание

Класс для логирования событий сервера

Автор

Гришин Николай

Версия

1.0

Дата

24.11.2025

Авторство

ИБСТ ПГУ

См. определение в файле [Logger.h](#) строка 28

3.5.2 Конструктор(ы)

3.5.2.1 `Logger()`

```
Logger::Logger (
    const std::string & filename )
```

Конструктор класса `Logger`.

Аргументы

<code>filename</code>	Имя файла для логирования
-----------------------	---------------------------

Исключения

<code>std::runtime_error</code>	если файл не может быть открыт
---------------------------------	--------------------------------

См. определение в файле `Logger.cpp` строка 10

3.5.2.2 `~Logger()`

```
Logger::~Logger ( )
```

Деструктор класса `Logger`.

См. определение в файле `Logger.cpp` строка 20

3.5.3 Методы

3.5.3.1 `error()`

```
void Logger::error (
    const std::string & message ) [inline]
```

Логирование ошибки

Аргументы

<code>message</code>	Сообщение для записи
----------------------	----------------------

См. определение в файле [Logger.h](#) строка [65](#)

3.5.3.2 getCurrentTime()

```
std::string Logger::getCurrentTime ( ) [private]
```

Получение текущего времени в формате строки

Возвращает

Строка с текущим временем

См. определение в файле [Logger.cpp](#) строка [47](#)

3.5.3.3 info()

```
void Logger::info (
    const std::string & message ) [inline]
```

Логирование информационного сообщения

Аргументы

message	Сообщение для записи
---------	----------------------

См. определение в файле [Logger.h](#) строка [53](#)

3.5.3.4 levelToString()

```
std::string Logger::levelToString (
    LogLevel level ) [private]
```

Преобразование уровня логирования в строку

Аргументы

level	Уровень логирования
-------	---------------------

Возвращает

Строковое представление уровня

См. определение в файле [Logger.cpp](#) строка 71

3.5.3.5 log()

```
void Logger::log (  
    LogLevel level,  
    const std::string & message )
```

Основной метод логирования

Аргументы

level	Уровень логирования
message	Сообщение для записи

См. определение в файле [Logger.cpp](#) строка 31

3.5.3.6 warning()

```
void Logger::warning (  
    const std::string & message ) [inline]
```

Логирование предупреждения

Аргументы

message	Сообщение для записи
---------	----------------------

См. определение в файле [Logger.h](#) строка 59

3.5.4 Данные класса

3.5.4.1 logFile

```
std::ofstream Logger::logFile [private]
```

Файл для записи логов

См. определение в файле [Logger.h](#) строка 68

3.5.4.2 logMutex

std::mutex Logger::logMutex [private]

Мьютекс для синхронизации доступа к файлу

См. определение в файле [Logger.h](#) строка 69

Объявления и описания членов классов находятся в файлах:

- [Logger.h](#)
- [Logger.cpp](#)

3.6 Класс Server

Основной класс сервера

```
#include <Server.h>
```

Открытые члены

- [Server](#) (const std::string &[clientBaseFile](#), const std::string &[logFile](#), int [port](#))
Конструктор класса [Server](#).
- [~Server](#) ()
Деструктор класса [Server](#).
- bool [initialize](#) ()
Инициализация сервера
- void [run](#) ()
Запуск основного цикла сервера
- void [stop](#) ()
Остановка сервера

Закрытые члены

- void [cleanup](#) ()
Очистка ресурсов сервера

Закрытые данные

- std::string [clientBaseFile](#)
Имя файла базы клиентов
- std::string [logFile](#)
Имя файла журнала
- int [port](#)
Порт сервера
- int [serverSocket](#)
Сокет сервера
- bool [running](#)
Флаг работы сервера
- std::unique_ptr< [ClientBase](#) > [clientBase](#)
Указатель на базу клиентов
- std::unique_ptr< [Logger](#) > [logger](#)
Указатель на логгер

3.6.1 Подробное описание

Основной класс сервера

Автор

Гришин Николай

Версия

1.0

Дата

24.11.2025

Авторство

ИБСТ ПГУ

См. определение в файле [Server.h](#) строка 21

3.6.2 Конструктор(ы)

3.6.2.1 Server()

```
Server::Server (  
    const std::string & clientBaseFile,  
    const std::string & logFile,  
    int port )
```

Конструктор класса [Server](#).

Аргументы

clientBaseFile	Файл базы клиентов
logFile	Файл журнала
port	Порт для прослушивания

См. определение в файле [Server.cpp](#) строка 12

3.6.2.2 ~Server()

```
Server::~Server ( )
```

Деструктор класса [Server](#).

См. определение в файле [Server.cpp](#) строка [19](#)

3.6.3 Методы

3.6.3.1 cleanup()

```
void Server::cleanup ( ) [private]
```

Очистка ресурсов сервера

См. определение в файле [Server.cpp](#) строка [120](#)

3.6.3.2 initialize()

```
bool Server::initialize ( )
```

Инициализация сервера

Возвращает

`true` если инициализация успешна, `false` в противном случае

См. определение в файле [Server.cpp](#) строка [27](#)

3.6.3.3 run()

```
void Server::run ( )
```

Запуск основного цикла сервера

См. определение в файле [Server.cpp](#) строка [84](#)

3.6.3.4 stop()

```
void Server::stop ( )
```

Остановка сервера

См. определение в файле [Server.cpp](#) строка [108](#)

3.6.4 Данные класса

3.6.4.1 clientBase

`std::unique_ptr<ClientBase> Server::clientBase [private]`

Указатель на базу клиентов

См. определение в файле [Server.h](#) строка 59

3.6.4.2 clientBaseFile

`std::string Server::clientBaseFile [private]`

Имя файла базы клиентов

См. определение в файле [Server.h](#) строка 53

3.6.4.3 logFile

`std::string Server::logFile [private]`

Имя файла журнала

См. определение в файле [Server.h](#) строка 54

3.6.4.4 logger

`std::unique_ptr<Logger> Server::logger [private]`

Указатель на логгер

См. определение в файле [Server.h](#) строка 60

3.6.4.5 port

`int Server::port [private]`

Порт сервера

См. определение в файле [Server.h](#) строка 55

3.6.4.6 running

```
bool Server::running [private]
```

Флаг работы сервера

См. определение в файле [Server.h](#) строка 57

3.6.4.7 serverSocket

```
int Server::serverSocket [private]
```

Сокет сервера

См. определение в файле [Server.h](#) строка 56

Объявления и описания членов классов находятся в файлах:

- [Server.h](#)
- [Server.cpp](#)

3.7 Структура ServerConfig

Структура для хранения конфигурации сервера

```
#include <CommandLineParser.h>
```

Открытые атрибуты

- std::string [clientBaseFile](#)
Файл базы клиентов
- std::string [logFile](#)
Файл журнала
- int [port](#)
Порт сервера

3.7.1 Подробное описание

Структура для хранения конфигурации сервера

См. определение в файле [CommandLineParser.h](#) строка 9

3.7.2 Данные класса

3.7.2.1 clientBaseFile

```
std::string ServerConfig::clientBaseFile
```

Файл базы клиентов

См. определение в файле [CommandLineParser.h](#) строка 10

3.7.2.2 logFile

```
std::string ServerConfig::logFile
```

Файл журнала

См. определение в файле [CommandLineParser.h](#) строка 11

3.7.2.3 port

```
int ServerConfig::port
```

Порт сервера

См. определение в файле [CommandLineParser.h](#) строка 12

Объявления и описания членов структуры находятся в файле:

- [CommandLineParser.h](#)

3.8 Класс VectorCalculator

Класс для вычисления произведения элементов вектора

```
#include <VectorCalculator.h>
```

Открытые статические члены

- `static uint64_t computeProduct (const std::vector< uint64_t > &vector)`

Вычисление произведения элементов вектора

3.8.1 Подробное описание

Класс для вычисления произведения элементов вектора

Автор

Гришин Николай

Версия

1.0

Дата

24.11.2025

Авторство

ИБСТ ПГУ

См. определение в файле [VectorCalculator.h](#) строка 14

3.8.2 Методы

3.8.2.1 computeProduct()

```
uint64_t VectorCalculator::computeProduct (
    const std::vector< uint64_t > & vector ) [static]
```

Вычисление произведения элементов вектора

Аргументы

vector	Вектор значений для вычисления
--------	--------------------------------

Возвращает

Произведение элементов вектора

Заметки

При переполнении возвращается UINT64_MAX

См. определение в файле [VectorCalculator.cpp](#) строка 12

Объявления и описания членов классов находятся в файлах:

- [VectorCalculator.h](#)
- [VectorCalculator.cpp](#)

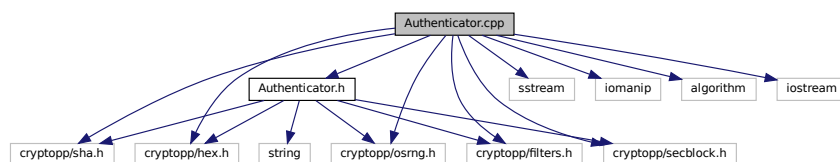
Глава 4

Файлы

4.1 Файл Authenticator.cpp

```
#include "Authenticator.h"  
#include <cryptopp/sha.h>  
#include <cryptopp/hex.h>  
#include <cryptopp/osrng.h>  
#include <cryptopp/filters.h>  
#include <cryptopp/secblock.h>  
#include <sstream>  
#include <iomanip>  
#include <algorithm>  
#include <iostream>
```

Граф включаемых заголовочных файлов для Authenticator.cpp:



4.2 Authenticator.cpp

[См. документацию.](#)

```
00001 #include "Authenticator.h"  
00002 #include <cryptopp/sha.h>  
00003 #include <cryptopp/hex.h>  
00004 #include <cryptopp/osrng.h>  
00005 #include <cryptopp/filters.h>  
00006 #include <cryptopp/secblock.h>  
00007 #include <sstream>  
00008 #include <iomanip>  
00009 #include <algorithm>  
00010 #include <iostream>  
00011  
00012 using namespace CryptoPP;  
00013  
00018 std::string Authenticator::generateSalt() {  
00019     // Используем генератор псевдослучайных чисел из Crypto++
```

```

00020     AutoSeededRandomPool prng;
00021
00022     // Генерируем 8 байт (64 бита) для соли
00023     byte saltBytes[8];
00024     prng.GenerateBlock(saltBytes, sizeof(saltBytes));
00025
00026     // Преобразуем в шестнадцатеричную строку
00027     std::string saltHex;
00028     HexEncoder encoder;
00029     encoder.Attach(new StringSink(saltHex));
00030     encoder.Put(saltBytes, sizeof(saltBytes));
00031     encoder.MessageEnd();
00032
00033     // Дополняем до 16 символов если нужно
00034     while (saltHex.length() < 16) {
00035         saltHex = "0" + saltHex;
00036     }
00037
00038     // Обрезаем до 16 символов если длиннее
00039     if (saltHex.length() > 16) {
00040         saltHex = saltHex.substr(0, 16);
00041     }
00042
00043     return saltHex;
00044 }
00045
00051 std::string Authenticator::computeSHA1(const std::string& data) {
00052     SHA1 hash;
00053     std::string digest;
00054
00055     // Используем цепочку фильтров для вычисления хеша
00056     StringSource(data, true,
00057         new HashFilter(hash,
00058             new HexEncoder(
00059                 new StringSink(digest))
00060         )
00061     );
00062
00063     return digest;
00064 }
00065
00073 bool Authenticator::authenticate(const std::string& password, const std::string& salt, const std::string& clientHash) {
00074     std::string combined = salt + password;
00075     std::string serverHash = computeSHA1(combined);
00076
00077     if (serverHash.empty()) {
00078         return false;
00079     }
00080
00081     // Сравниваем хеши (без учета регистра)
00082     bool match = true;
00083     for (size_t i = 0; i < serverHash.length() && i < clientHash.length(); i++) {
00084         char serverChar = std::tolower(serverHash[i]);
00085         char clientChar = std::tolower(clientHash[i]);
00086
00087         if (serverChar != clientChar) {
00088             match = false;
00089             break;
00090         }
00091     }
00092
00093     return match;
00094 }

```

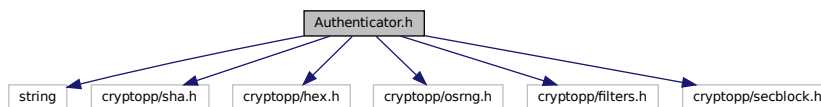
4.3 Файл Authenticator.h

```

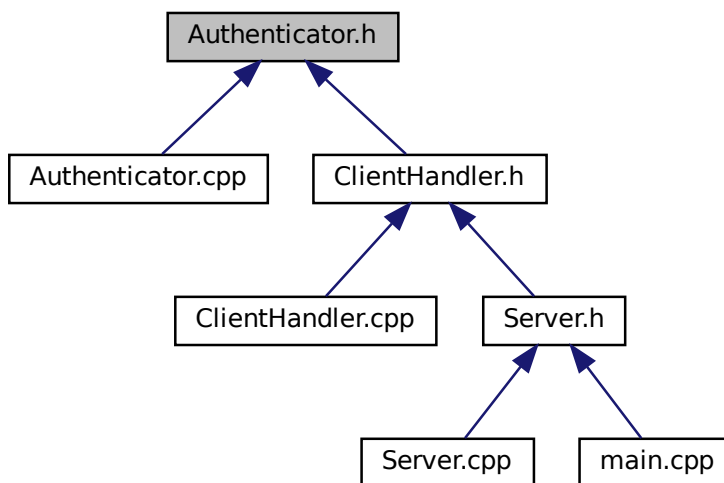
#include <string>
#include <cryptopp/sha.h>
#include <cryptopp/hex.h>
#include <cryptopp/osrng.h>
#include <cryptopp/filters.h>
#include <cryptopp/secblock.h>

```

Граф включаемых заголовочных файлов для Authenticator.h:



Граф файлов, в которые включается этот файл:



Классы

- class [Authenticator](#)

Класс для аутентификации клиентов с использованием Crypto++.

4.4 Authenticator.h

См. документацию.

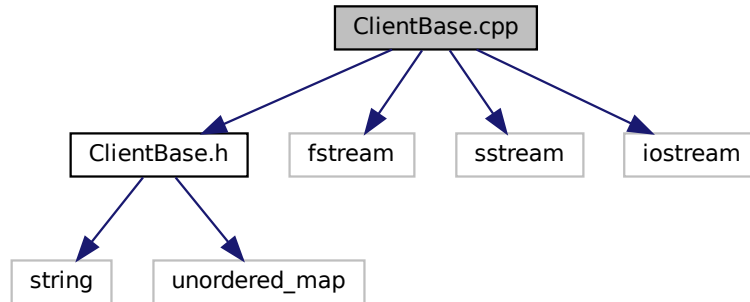
```

00001 #ifndef AUTHENTICATOR_H
00002 #define AUTHENTICATOR_H
00003
00004 #include <string>
00005 #include <cryptopp/sha.h>
00006 #include <cryptopp/hex.h>
00007 #include <cryptopp/osrng.h>
00008 #include <cryptopp/filters.h>
00009 #include <cryptopp/secblock.h>
00010
00018 class Authenticator {
00019 public:
00024     static std::string generateSalt();
00025
00031     static std::string computeSHA1(const std::string& data);
00032
00040     static bool authenticate(const std::string& password, const std::string& salt, const std::string& clientHash);
00041 };
00042
00043 #endif
  
```

4.5 Файл ClientBase.cpp

```
#include "ClientBase.h"
#include <fstream>
#include <sstream>
#include <iostream>
```

Граф включаемых заголовочных файлов для ClientBase.cpp:



4.6 ClientBase.cpp

[См. документацию.](#)

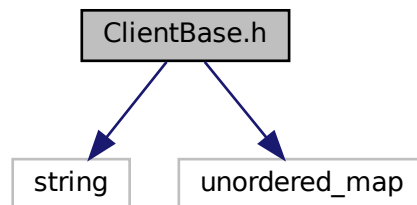
```
00001 #include "ClientBase.h"
00002 #include <fstream>
00003 #include <sstream>
00004 #include <iostream>
00005
00010 ClientBase::ClientBase(const std::string& filename) : filename(filename) {}
00011
00016 bool ClientBase::load() {
00017     std::ifstream file(filename);
00018     if (!file.is_open()) {
00019         return false;
00020     }
00021
00022     std::string line;
00023     while (std::getline(file, line)) {
00024         size_t pos = line.find(':');
00025         if (pos != std::string::npos) {
00026             std::string login = line.substr(0, pos);
00027             std::string password = line.substr(pos + 1);
00028             clients[login] = password;
00029         }
00030     }
00031
00032     file.close();
00033     return true;
00034 }
00035
00041 bool ClientBase::userExists(const std::string& login) const {
00042     return clients.find(login) != clients.end();
00043 }
00044
00050 std::string ClientBase::getPassword(const std::string& login) const {
00051     auto it = clients.find(login);
00052     return (it != clients.end()) ? it->second : "";
00053 }
```

4.7 Файл ClientBase.h

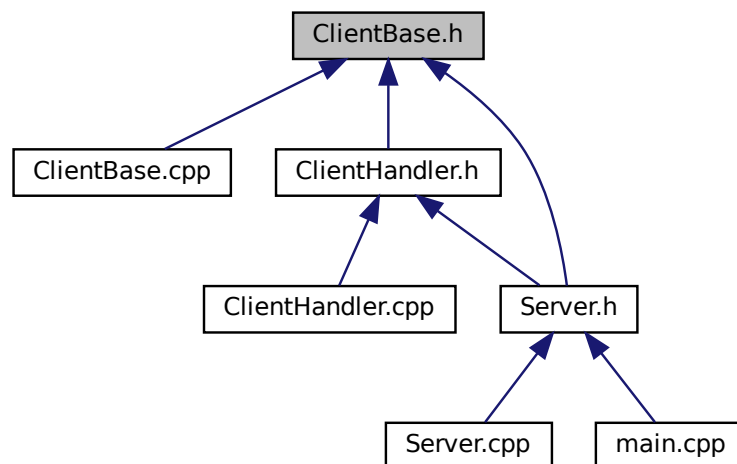
```
#include <string>
```

```
#include <unordered_map>
```

Граф включаемых заголовочных файлов для ClientBase.h:



Граф файлов, в которые включается этот файл:



Классы

- class `ClientBase`

Класс для работы с базой данных клиентов

4.9.1.2 htobe64

```
#define htobe64(
    x ) __bswap_64(x)
```

См. определение в файле [ClientHandler.cpp](#) строка 11

4.10 ClientHandler.cpp

См. документацию.

```
00001 #include "ClientHandler.h"
00002 #include <iostream>
00003 #include <string>
00004 #include <cstring>
00005 #include <arpa/inet.h> // для be64toh/htobe64
00006
00007 #ifndef be64toh
00008 // Если макросы не определены, создаем свои
00009 #if __BYTE_ORDER == __LITTLE_ENDIAN
00010 #define be64toh(x) __bswap_64(x)
00011 #define htobe64(x) __bswap_64(x)
00012 #else
00013 #define be64toh(x) (x)
00014 #define htobe64(x) (x)
00015 #endif
00016 #endif
00017
00025 ClientHandler::ClientHandler(int clientSocket, const sockaddr_in& clientAddr,
00026                               ClientBase& clientBase, Logger& logger)
00027 : clientSocket(clientSocket), clientAddr(clientAddr),
00028   clientBase(clientBase), logger(logger) {}
00029
00034 void ClientHandler::handle() {
00035     std::string clientIP = inet_ntoa(clientAddr.sin_addr);
00036     int clientPort = ntohs(clientAddr.sin_port);
00037     logger.info("Client connected from " + clientIP + ":" + std::to_string(clientPort));
00038
00039     try {
00040         if (!authenticateClient()) {
00041             logger.warning("Authentication failed for client " + clientIP);
00042             close(clientSocket);
00043             return;
00044         }
00045         logger.info("Client " + clientIP + " authenticated successfully");
00046
00047         if (!receiveVectorsAndProcess()) {
00048             logger.warning("Vector processing failed for client " + clientIP);
00049         }
00050     } catch (const std::exception& e) {
00051         logger.error("Error handling client " + clientIP + ": " + e.what());
00052     }
00053     close(clientSocket);
00054     logger.info("Client " + clientIP + " disconnected");
00055 }
00056
00071 bool ClientHandler::authenticateClient() {
00072     // Шаг 1: Получаем логин
00073     std::string login;
00074     if (!receiveString(login)) {
00075         logger.error("Failed to receive login");
00076         return false;
00077     }
00078
00079     // Удаляем все пробелы из имени пользователя
00080     login.erase(0, login.find_first_not_of(" \t\n\r"));
00081     login.erase(login.find_last_not_of(" \t\n\r") + 1);
00082
00083     logger.info("Received login: '" + login + "'");
00084
00085     // Шаг 2: Проверяем, существует ли пользователь
00086     if (!clientBase.userExists(login)) {
00087         logger.warning("User not found: " + login);
00088         sendMessage("ERR");
00089         return false;
00090     }
00091 }
```

```

00090     }
00091
00092     // ИСПОЛЬЗУЕМ ПАРОЛЬ ИЗ БАЗЫ ДАННЫХ
00093     std::string password = clientBase.getPassword(login);
00094
00095     // Шаг 3: Генерируем и отправляем соль с помощью Crypto++
00096     std::string salt = Authenticator::generateSalt();
00097     logger.info("Generated salt for user " + login + ": " + salt);
00098
00099     if (!sendSalt(salt)) {
00100         logger.error("Failed to send salt");
00101         return false;
00102     }
00103
00104     // Шаг 4: Получаем хэш от клиента
00105     std::string clientHash;
00106
00107     // Используем receiveString для хеша тоже
00108     if (!receiveString(clientHash)) {
00109         logger.error("Failed to receive hash");
00110         return false;
00111     }
00112
00113     logger.info("Received hash from client");
00114
00115     // Шаг 5: Аутентификация с помощью Crypto++
00116     bool authResult = Authenticator::authenticate(password, salt, clientHash);
00117
00118     if (authResult) {
00119         logger.info("Authentication successful for user: " + login);
00120         return sendMessage("OK");
00121     } else {
00122         logger.warning("Authentication failed for user: " + login);
00123         sendMessage("ERR");
00124         return false;
00125     }
00126 }
00127
00139 bool ClientHandler::receiveVectorsAndProcess() {
00140     // Шаг 6: Получаем количество векторов
00141     uint32_t numVectors;
00142     if (!receiveData(&numVectors, sizeof(numVectors))) {
00143         logger.error("Failed to receive number of vectors");
00144         return false;
00145     }
00146
00147     numVectors = le32toh(numVectors);
00148
00149     if (numVectors > 1000) {
00150         logger.error("Invalid number of vectors: " + std::to_string(numVectors));
00151         return false;
00152     }
00153
00154     // НЕ отправляем количество результатов - это не результат вычислений
00155     // Просто переходим к обработке векторов
00156
00157     // Обрабатываем каждый вектор
00158     for (uint32_t i = 0; i < numVectors; i++) {
00159         // Шаг 7: Получаем размер вектора
00160         uint32_t vectorSize;
00161         if (!receiveData(&vectorSize, sizeof(vectorSize))) {
00162             logger.error("Failed to receive vector size");
00163             return false;
00164         }
00165         vectorSize = le32toh(vectorSize);
00166
00167         if (vectorSize > 100000) {
00168             logger.error("Invalid vector size: " + std::to_string(vectorSize));
00169             return false;
00170         }
00171
00172         // Шаг 8: Получаем данные вектора
00173         std::vector<uint64_t> vector(vectorSize);
00174         for (uint32_t j = 0; j < vectorSize; j++) {
00175             uint64_t value;
00176             if (!receiveData(&value, sizeof(value))) {
00177                 logger.error("Failed to receive vector data");
00178                 return false;
00179             }
00180             value = le64toh(value);
00181             vector[j] = value;
00182         }
00183
00184         // Шаг 9: Вычисляем и отправляем результат
00185         uint64_t result = VectorCalculator::computeProduct(vector);
00186
00187         result = htobe64(result);

```

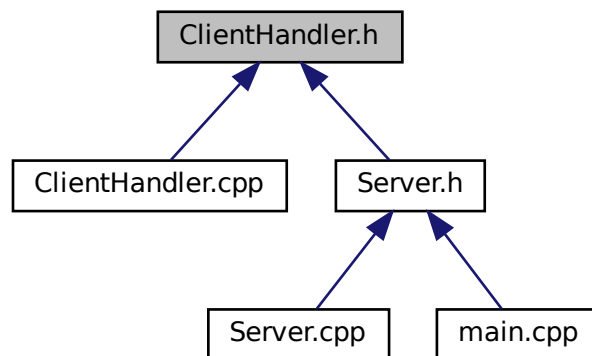


```

00188         if (!sendData(&result, sizeof(result))) {
00189             logger.error("Failed to send result");
00190             return false;
00191         }
00192     }
00193
00194     return true;
00195 }
00196
00203 bool ClientHandler::sendSalt(const std::string& salt) {
00204     // Для соли отправляем БЕЗ нулевого байта (16 байт)
00205     return sendData(salt.c_str(), 16); // Только 16 байт
00206 }
00207
00214 bool ClientHandler::sendMessage(const std::string& message) {
00215     // Для текстовых сообщений отправляем С нулевым байтом
00216     return sendData(message.c_str(), message.length() + 1);
00217 }
00218
00226 bool ClientHandler::receiveData(void* buffer, size_t size) {
00227     // Для бинарных данных всегда читаем точное количество байт
00228     // без поиска нулевого терминатора
00229
00230     size_t totalReceived = 0;
00231     while (totalReceived < size) {
00232         ssize_t received = recv(clientSocket,
00233                                static_cast<char*>(buffer) + totalReceived,
00234                                size - totalReceived, 0);
00235
00236         if (received <= 0) {
00237             return false;
00238         }
00239         totalReceived += received;
00240     }
00241
00242     return true;
00243 }
00244
00252 bool ClientHandler::sendData(const void* data, size_t size) {
00253     const char* charData = static_cast<const char*>(data);
00254
00255     size_t totalSent = 0;
00256     while (totalSent < size) {
00257         ssize_t sent = send(clientSocket,
00258                             static_cast<const char*>(data) + totalSent,
00259                             size - totalSent, 0);
00260         if (sent <= 0) {
00261             return false;
00262         }
00263         totalSent += sent;
00264     }
00265     return true;
00266 }
00267
00278 bool ClientHandler::receiveString(std::string& result, size_t maxSize) {
00279     char buffer[256] = {0};
00280     size_t totalReceived = 0;
00281
00282     // Сначала посмотрим, что находится в буфере
00283     char peekBuffer[256] = {0};
00284     ssize_t peeked = recv(clientSocket, peekBuffer, sizeof(peekBuffer) - 1, MSG_PEEK);
00285
00286     if (peeked > 0) {
00287
00288         // Если мы видим полную строку без терминатора, считываем ровно столько байт
00289         bool hasTerminator = false;
00290         for (int i = 0; i < peeked; i++) {
00291             if (peekBuffer[i] == '\0' || peekBuffer[i] == '\n' || peekBuffer[i] == '\r') {
00292                 hasTerminator = true;
00293                 break;
00294             }
00295         }
00296     }
00297
00298     if (!hasTerminator && peeked > 0) {
00299         // Терминатор не найден, прочитаем ровно столько байт, сколько мы просмотрели
00300         ssize_t received = recv(clientSocket, buffer, peeked, 0);
00301         if (received == peeked) {
00302             buffer[received] = '\0'; // Добавляем нулевой терминатор
00303             result = std::string(buffer);
00304             return true;
00305         }
00306     }
00307 }
00308
00309 // Резервный вариант: читаем побайтно
00310 while (totalReceived < maxSize - 1) {

```


Граф файлов, в которые включается этот файл:



Классы

- class [ClientHandler](#)

Класс для обработки клиентских подключений

4.12 ClientHandler.h

[См. документацию.](#)

```

00001 #ifndef CLIENTHANDLER_H
00002 #define CLIENTHANDLER_H
00003
00004 #include "ClientBase.h"
00005 #include "Logger.h"
00006 #include "Authenticator.h"
00007 #include "VectorCalculator.h"
00008 #include <sys/socket.h>
00009 #include <netinet/in.h>
00010 #include <unistd.h>
00011 #include <arpa/inet.h>
00012 #include <vector>
00013
00021 class ClientHandler {
00022 public:
00030     ClientHandler(int clientSocket, const sockaddr_in& clientAddr,
00031                  ClientBase& clientBase, Logger& logger);
00032
00036     void handle();
00037
00038 private:
00039     int clientSocket;
00040     sockaddr_in clientAddr;
00041     ClientBase& clientBase;
00042     Logger& logger;
00043
00048     bool authenticateClient();
00049
00054     bool receiveVectorsAndProcess();
00055
00061     bool sendMessage(const std::string& message);
00062
00069     bool receiveData(void* buffer, size_t size);
00070
00077     bool sendData(const void* data, size_t size);
00078
00085     bool receiveString(std::string& result, size_t maxSize = 256);
  
```

```

00086
00092     bool sendSalt(const std::string& salt);
00093 };
00094
00095 #endif

```

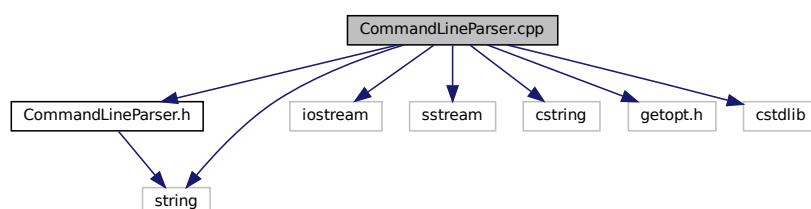
4.13 Файл CommandLineParser.cpp

```

#include "CommandLineParser.h"
#include <iostream>
#include <sstream>
#include <string>
#include <cstring>
#include <getopt.h>
#include <cstdlib>

```

Граф включаемых заголовочных файлов для CommandLineParser.cpp:



4.14 CommandLineParser.cpp

См. документацию.

```

00001 #include "CommandLineParser.h"
00002 #include <iostream>
00003 #include <sstream>
00004 #include <string>
00005 #include <cstring>
00006 #include <getopt.h>
00007 #include <cstdlib>
00008
00009 using namespace std;
00010
00014 CommandLineParser::CommandLineParser() {
00015     config.port = 0;
00016     config.clientBaseFile = "";
00017     config.logFile = "";
00018
00019     std::ostringstream oss;
00020     oss << "Server usage:\n"
00021         << "  -b, --base <file>    Client database file (REQUIRED)\n"
00022         << "  -l, --log <file>      Log file (REQUIRED)\n"
00023         << "  -p, --port <port>    Port to listen on (REQUIRED)\n"
00024         << "  -h, --help          Show this help\n"
00025         << "Example: ./server -b clients.txt -l server.log -p 33333\n";
00026     helpText = oss.str();
00027 }
00028
00033 bool CommandLineParser::validateRequiredParams() const {
00034     if (config.clientBaseFile.empty()) {
00035         cerr << "Error: Client database file is required (-b, --base)" << endl;
00036         return false;
00037     }
00038
00039     if (config.logFile.empty()) {
00040         cerr << "Error: Log file is required (-l, --log)" << endl;
00041         return false;

```

```

00042     }
00043
00044     if (config.port == 0) {
00045         cerr << "Error: Port is required (-p, --port)" << endl;
00046         return false;
00047     }
00048
00049     return true;
00050 }
00051
00052 bool CommandLineParser::parse(int argc, char* argv[]) {
00053     // Сбрасываем конфиг перед парсингом
00054     config.port = 0;
00055     config.clientBaseFile = "";
00056     config.logFile = "";
00057
00058     // Сбрасываем optind для повторных вызовов getopt_long
00059     optind = 1;
00060
00061     // Структура для длинных опций getopt_long
00062     static struct option long_options[] = {
00063         {"base", required_argument, 0, 'b'}, // Файл базы клиентов
00064         {"log", required_argument, 0, 'l'}, // Файл журнала
00065         {"port", required_argument, 0, 'p'}, // Порт сервера
00066         {"help", no_argument, 0, 'h'}, // Справка
00067         {0, 0, 0, 0} // Конец массива
00068     };
00069
00070     // Строка коротких опций для getopt
00071     const char* short_options = "b:lp:h";
00072
00073     // Разбор опций с помощью getopt_long
00074     int opt;
00075     int option_index = 0;
00076
00077     // Используем цикл для обработки всех опций
00078     while ((opt = getopt_long(argc, argv, short_options, long_options, &option_index)) != -1) {
00079         switch (opt) {
00080             case 'b': // Файл базы клиентов
00081                 config.clientBaseFile = optarg;
00082                 break;
00083
00084             case 'l': // Файл журнала
00085                 config.logFile = optarg;
00086                 break;
00087
00088             case 'p': // Порт сервера
00089                 try {
00090                     config.port = stoi(string(optarg));
00091
00092                     // Проверяем допустимость порта
00093                     if (config.port < 1 || config.port > 65535) {
00094                         cerr << "Error: Port must be between 1 and 65535" << endl;
00095                         return false;
00096                     }
00097                 } catch (const exception&) {
00098                     cerr << "Error: Invalid port number: " << optarg << endl;
00099                     return false;
00100                 }
00101                 break;
00102
00103             case 'h': // Справка
00104                 cout << helpText;
00105                 return false; // Завершаем выполнение после вывода справки
00106
00107             case '?': // Неизвестная опция
00108                 // getopt_long уже выводит сообщение об ошибке
00109                 cerr << "\nUse -h or --help for usage information" << endl;
00110                 return false;
00111
00112             case ':': // Отсутствует аргумент для опции
00113                 cerr << "Error: Missing argument for option '"
00114                     << static_cast<char>(optopt) << "' " << endl;
00115                 cerr << "Use -h or --help for usage information" << endl;
00116                 return false;
00117
00118             default:
00119                 cerr << "Error: Unknown error parsing parameters" << endl;
00120                 return false;
00121         }
00122     }
00123
00124     // Проверяем наличие лишних аргументов (не опций)
00125     if (optind < argc) {
00126         cerr << "Error: Unknown arguments: ";
00127         for (int i = optind; i < argc; i++) {
00128             cerr << argv[i] << " ";
00129         }
00130     }

```

```

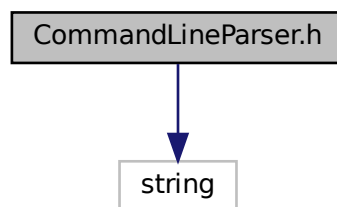
00135     }
00136     cerr << "\nUse -h or --help for usage information" << endl;
00137     return false;
00138 }
00139
00140 // Проверяем обязательные параметры
00141 if (!validateRequiredParams()) {
00142     cerr << "Use -h or --help for usage information" << endl;
00143     return false;
00144 }
00145
00146 return true;
00147 }
00148
00153 std::string CommandLineParser::getHelp()const {
00154     return helpText;
00155 }

```

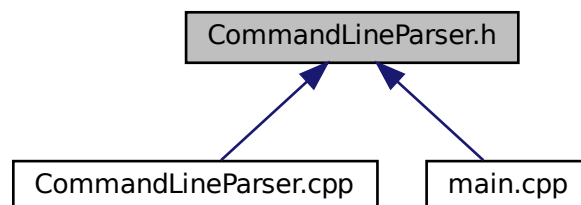
4.15 Файл CommandLineParser.h

```
#include <string>
```

Граф включаемых заголовочных файлов для CommandLineParser.h:



Граф файлов, в которые включается этот файл:



Классы

- struct [ServerConfig](#)
Структура для хранения конфигурации сервера
- class [CommandLineParser](#)
Класс для разбора командной строки с использованием getopt/getopt_long.

4.16 CommandLineParser.h

[См. документацию.](#)

```

00001 #ifndef COMMANDLINEPARSER_H
00002 #define COMMANDLINEPARSER_H
00003
00004 #include <string>
00005
00009 struct ServerConfig {
00010     std::string clientBaseFile;
00011     std::string logFile;
00012     int port;
00013 };
00014
00022 class CommandLineParser {
00023 public:
00027     CommandLineParser();
00028
00035     bool parse(int argc, char* argv[]);
00036
00041     ServerConfig getConfig()const { return config; }
00042
00047     std::string getHelp() const;
00048
00049 private:
00050     ServerConfig config;
00051     std::string helpText;
00052
00057     bool validateRequiredParams() const;
00058 };
00059
00060 #endif

```

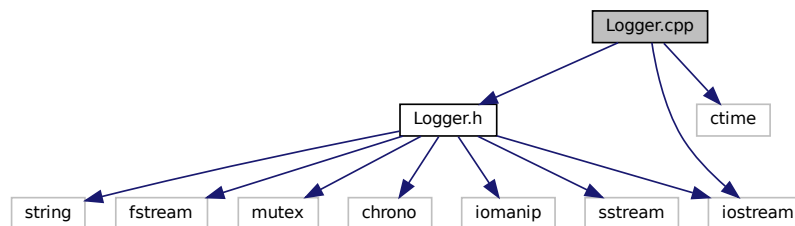
4.17 Файл Logger.cpp

```

#include "Logger.h"
#include <iostream>
#include <ctime>

```

Граф включаемых заголовочных файлов для Logger.cpp:



4.18 Logger.cpp

[См. документацию.](#)

```

00001 #include "Logger.h"
00002 #include <iostream>
00003 #include <ctime>
00004
00010 Logger::Logger(const std::string& filename) {
00011     logFile.open(filename, std::ios::app);
00012     if (!logFile.is_open()) {
00013         throw std::runtime_error("Cannot open log file: " + filename);
00014     }

```

```

00015 }
00016
00020 Logger::~Logger() {
00021     if (logFile.is_open()) {
00022         logFile.close();
00023     }
00024 }
00025
00031 void Logger::log(LogLevel level, const std::string& message) {
00032     std::lock_guard<std::mutex> lock(logMutex);
00033
00034     std::string logEntry = getCurrentTime() + " [" + levelToString(level) + " ] " + message + "\n";
00035
00036     logFile << logEntry;
00037     logFile.flush();
00038
00039     // Also output to console
00040     std::cout << logEntry;
00041 }
00042
00047 std::string Logger::getCurrentTime() {
00048     auto now = std::chrono::system_clock::now();
00049     auto time_t = std::chrono::system_clock::to_time_t(now);
00050
00051     std::stringstream ss;
00052
00053     // Безопасная альтернатива: используем статический буфер с мьютексом
00054     static std::mutex time_mutex;
00055     std::tm tm_buffer;
00056
00057     {
00058         std::lock_guard<std::mutex> lock(time_mutex);
00059         tm_buffer = *std::localtime(&time_t);
00060     }
00061
00062     ss << std::put_time(&tm_buffer, "%Y-%m-%d %H:%M:%S");
00063     return ss.str();
00064 }
00065
00071 std::string Logger::levelToString(LogLevel level) {
00072     switch (level) {
00073         case LogLevel::INFO: return "INFO";
00074         case LogLevel::WARNING: return "WARNING";
00075         case LogLevel::ERROR: return "ERROR";
00076         default: return "UNKNOWN";
00077     }
00078 }

```

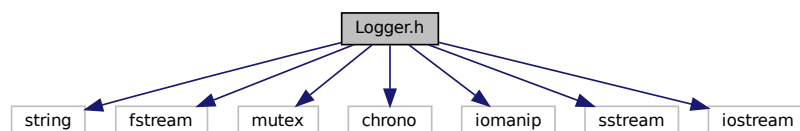
4.19 Файл Logger.h

```

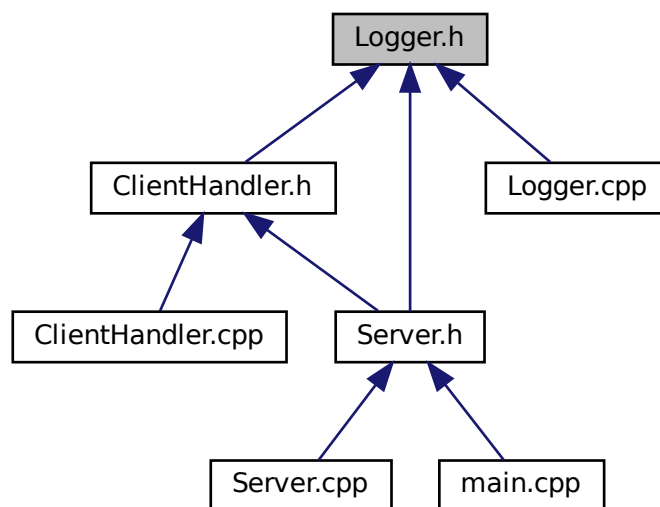
#include <string>
#include <fstream>
#include <mutex>
#include <chrono>
#include <iomanip>
#include <sstream>
#include <iostream>

```

Граф включаемых заголовочных файлов для Logger.h:



Граф файлов, в которые включается этот файл:



Классы

- class `Logger`
Класс для логирования событий сервера

Перечисления

- enum class `LogLevel` { `INFO` , `WARNING` , `ERROR` }
Перечисление уровней логирования

4.19.1 Перечисления

4.19.1.1 LogLevel

enum class `LogLevel` [strong]

Перечисление уровней логирования

Элементы перечислений

INFO	Информационное сообщение
WARNING	Предупреждение
ERROR	Ошибка

См. определение в файле [Logger.h](#) строка 15

4.20 Logger.h

См. документацию.

```

00001 #ifndef LOGGER_H
00002 #define LOGGER_H
00003
00004 #include <string>
00005 #include <fstream>
00006 #include <mutex>
00007 #include <chrono>
00008 #include <iomanip>
00009 #include <sstream>
00010 #include <iostream>
00011
00015 enum class LogLevel {
00016     INFO,
00017     WARNING,
00018     ERROR
00019 };
00020
00028 class Logger {
00029 public:
00035     Logger(const std::string& filename);
00036
00040     ~Logger();
00041
00047     void log(LogLevel level, const std::string& message);
00048
00053     void info(const std::string& message) { log(LogLevel::INFO, message); }
00054
00059     void warning(const std::string& message) { log(LogLevel::WARNING, message); }
00060
00065     void error(const std::string& message) { log(LogLevel::ERROR, message); }
00066
00067 private:
00068     std::ofstream logFile;
00069     std::mutex logMutex;
00070
00075     std::string getCurrentTime();
00076
00082     std::string levelToString(LogLevel level);
00083 };
00084
00085 #endif

```

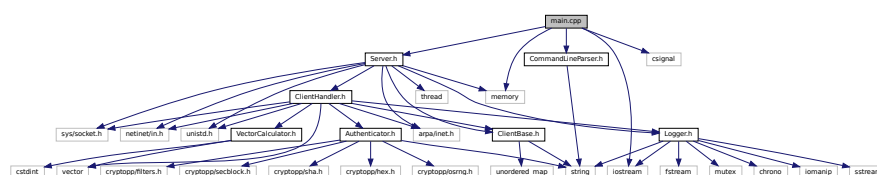
4.21 Файл main.cpp

```

#include "CommandLineParser.h"
#include "Server.h"
#include <iostream>
#include <csignal>
#include <memory>

```

Граф включаемых заголовочных файлов для main.cpp:



Функции

- void [signalHandler](#) (int signal)
Обработчик сигналов для graceful shutdown.
- int [main](#) (int argc, char *argv[])
Главная функция сервера

Переменные

- std::unique_ptr< [Server](#) > [server](#)
Глобальный указатель на сервер для обработки сигналов

4.21.1 Функции

4.21.1.1 main()

```
int main (  
    int argc,  
    char * argv[] )
```

Главная функция сервера

Аргументы

argc	Количество аргументов командной строки
argv	Массив аргументов командной строки

Возвращает

Код завершения программы

Автор

Гришин Николай

Версия

1.0

Дата

24.11.2025

Авторство

ИБСТ ПГУ

См. определение в файле [main.cpp](#) строка [30](#)

4.21.1.2 signalHandler()

```
void signalHandler (
    int signal )
```

Обработчик сигналов для graceful shutdown.

Аргументы

signal	Номер сигнала
--------	---------------

См. определение в файле [main.cpp](#) строка 13

4.21.2 Переменные

4.21.2.1 server

```
std::unique_ptr<Server> server
```

Глобальный указатель на сервер для обработки сигналов

См. определение в файле [main.cpp](#) строка 7

4.22 main.cpp

См. документацию.

```
00001 #include "CommandLineParser.h"
00002 #include "Server.h"
00003 #include <iostream>
00004 #include <csignal>
00005 #include <memory>
00006
00007 std::unique_ptr<Server> server;
00008
00013 void signalHandler(int signal) {
00014     if (server) {
00015         std::cout << "\nShutting down server..." << std::endl;
00016         server->stop();
00017     }
00018 }
00019
00030 int main(int argc, char* argv[]) {
00031     // Настройка обработчиков сигналов
00032     signal(SIGINT, signalHandler);
00033     signal(SIGTERM, signalHandler);
00034
00035     // Разбор аргументов командной строки
00036     CommandLineParser parser;
00037     if (!parser.parse(argc, argv)) {
00038         return 0;
00039     }
00040
00041     ServerConfig config = parser.getConfig();
00042
00043     try {
00044         // Создание и инициализация сервера
00045         server = std::make_unique<Server>(config.clientBaseFile, config.logFile, config.port);
00046
00047         if (!server->initialize()) {
00048             std::cerr << "Failed to initialize server" << std::endl;
```

```

00049         return 1;
00050     }
00051
00052     // Запуск сервера
00053     server->run();
00054
00055 } catch (const std::exception& e) {
00056     std::cerr << "Server error: " << e.what() << std::endl;
00057     return 1;
00058 }
00059
00060 return 0;
00061 }

```

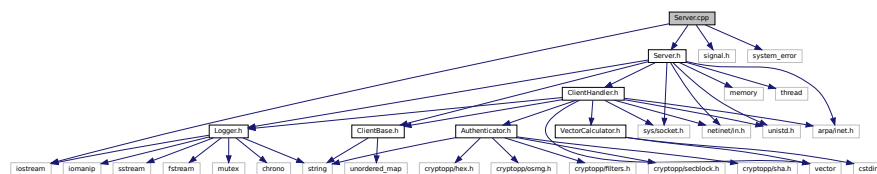
4.23 Файл Server.cpp

```

#include "Server.h"
#include <iostream>
#include <signal.h>
#include <system_error>

```

Граф включаемых заголовочных файлов для Server.cpp:



4.24 Server.cpp

См. документацию.

```

00001 #include "Server.h"
00002 #include <iostream>
00003 #include <signal.h>
00004 #include <system_error>
00005
00012 Server::Server(const std::string& clientBaseFile, const std::string& logFile, int port)
00013     : clientBaseFile(clientBaseFile), logFile(logFile), port(port),
00014       serverSocket(-1), running(false) {}
00015
00019 Server::~~Server() {
00020     cleanup();
00021 }
00022
00027 bool Server::initialize() {
00028     // Инициализация базы клиентов
00029     clientBase = std::make_unique<ClientBase>(clientBaseFile);
00030     if (!clientBase->load()) {
00031         std::cerr << "Failed to load client database: " << clientBaseFile << std::endl;
00032         return false;
00033     }
00034
00035     // ДОБАВИТЬ: проверим что база загрузилась
00036     std::cout << "DEBUG: Client database loaded successfully" << std::endl;
00037
00038     // Инициализация Логера
00039     try {
00040         logger = std::make_unique<Logger>(logFile);
00041     } catch (const std::exception& e) {
00042         std::cerr << "Failed to initialize logger: " << e.what() << std::endl;
00043         return false;
00044     }
00045
00046     // Создание сокета
00047     serverSocket = socket(AF_INET, SOCK_STREAM, 0);
00048     if (serverSocket == -1) {

```

```

00049     logger->error("Failed to create socket");
00050     return false;
00051 }
00052
00053 // Установка параметров сокета
00054 int opt = 1;
00055 if (setsockopt(serverSocket, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) == -1) {
00056     logger->error("Failed to set socket options");
00057     return false;
00058 }
00059
00060 // Привязка сокета
00061 sockaddr_in serverAddr{};
00062 serverAddr.sin_family = AF_INET;
00063 serverAddr.sin_addr.s_addr = INADDR_ANY;
00064 serverAddr.sin_port = htons(port);
00065
00066 if (bind(serverSocket, (struct sockaddr*)&serverAddr, sizeof(serverAddr)) == -1) {
00067     logger->error("Failed to bind socket to port " + std::to_string(port));
00068     return false;
00069 }
00070
00071 // Listen for connections
00072 if (listen(serverSocket, 10) == -1) {
00073     logger->error("Failed to listen on socket");
00074     return false;
00075 }
00076
00077 logger->info("Server initialized successfully on port " + std::to_string(port));
00078 return true;
00079 }
00080
00084 void Server::run() {
00085     running = true;
00086     logger->info("Server started and listening for connections");
00087
00088     while (running) {
00089         sockaddr_in clientAddr{};
00090         socklen_t clientAddrLen = sizeof(clientAddr);
00091
00092         int clientSocket = accept(serverSocket, (struct sockaddr*)&clientAddr, &clientAddrLen);
00093         if (clientSocket == -1) {
00094             if (running) {
00095                 logger->error("Failed to accept client connection");
00096             }
00097             continue;
00098         }
00099
00100         ClientHandler handler(clientSocket, clientAddr, *clientBase, *logger);
00101         handler.handle();
00102     }
00103 }
00104
00108 void Server::stop() {
00109     running = false;
00110     if (serverSocket != -1) {
00111         close(serverSocket);
00112         serverSocket = -1;
00113     }
00114     logger->info("Server stopped");
00115 }
00116
00120 void Server::cleanup() {
00121     stop();
00122 }

```

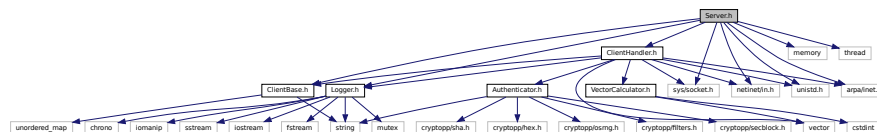
4.25 Файл Server.h

```

#include "ClientBase.h"
#include "Logger.h"
#include "ClientHandler.h"
#include <sys/socket.h>
#include <netinet/in.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <memory>

```

Граф включаемых заголовочных файлов для Server.h:



```
graph BT; Servercpp[Server.cpp] --> Serverh[Server.h]; maincpp[main.cpp] --> Serverh;
```

The diagram illustrates a header file, `Server.h`, which is shared by two source files, `Server.cpp` and `main.cpp`. Arrows point from both source files to the header file, indicating that they both include and use the definitions provided in `Server.h`.

- class **Server**
Основной класс сервера

См. документацию.

```
00001 #ifndef SERVER_H
00002 #define SERVER_H
00003
00004 #include "ClientBase.h"
00005 #include "Logger.h"
00006 #include "ClientHandler.h"
00007 #include <sys/socket.h>
00008 #include <netinet/in.h>
00009 #include <unistd.h>
00010 #include <arpa/inet.h>
00011 #include <memory>
00012 #include <thread>
00013
00021 class Server {
00022 public:
00029     Server(const std::string& clientBaseFile, const std::string& logFile, int port);
00030
00034     ~Server();
00035
00040     bool initialize();
00041
00045     void run();
00046
00050     void stop();
00051
00052 private:
00053     std::string clientBaseFile;
```

```

00054     std::string logFile;
00055     int port;
00056     int serverSocket;
00057     bool running;
00058
00059     std::unique_ptr<ClientBase> clientBase;
00060     std::unique_ptr<Logger> logger;
00061
00065     void cleanup();
00066 };
00067
00068 #endif

```

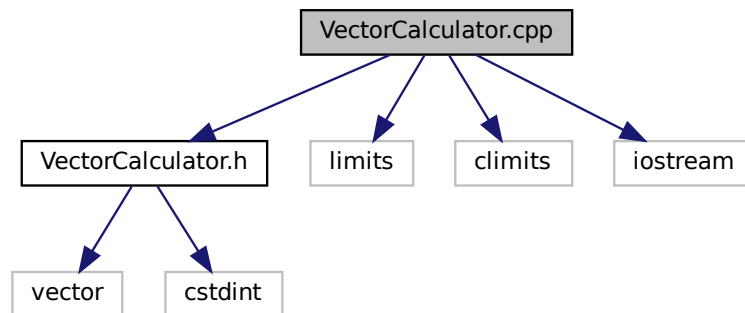
4.27 Файл VectorCalculator.cpp

```

#include "VectorCalculator.h"
#include <limits>
#include <climits>
#include <iostream>

```

Граф включаемых заголовочных файлов для VectorCalculator.cpp:



4.28 VectorCalculator.cpp

[См. документацию.](#)

```

00001 #include "VectorCalculator.h"
00002 #include <limits>
00003 #include <climits>
00004 #include <iostream>
00005
00012 uint64_t VectorCalculator::computeProduct(const std::vector<uint64_t>& vector) {
00013     if (vector.empty()) {
00014         return 0;
00015     }
00016
00017     uint64_t product = 1;
00018
00019     for (uint64_t value : vector) {
00020         if (value == 0) {
00021             return 0;
00022         }
00023
00024         // Проверка на переполнение перед умножением
00025         if (product > UINT64_MAX / value) {
00026             return UINT64_MAX; // 2^64 - 1
00027         }
00028         product *= value;
00029     }
00030
00031     return product;
00032 }
00033 }

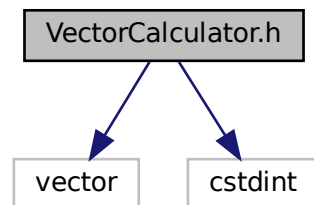
```


4.29 Файл VectorCalculator.h

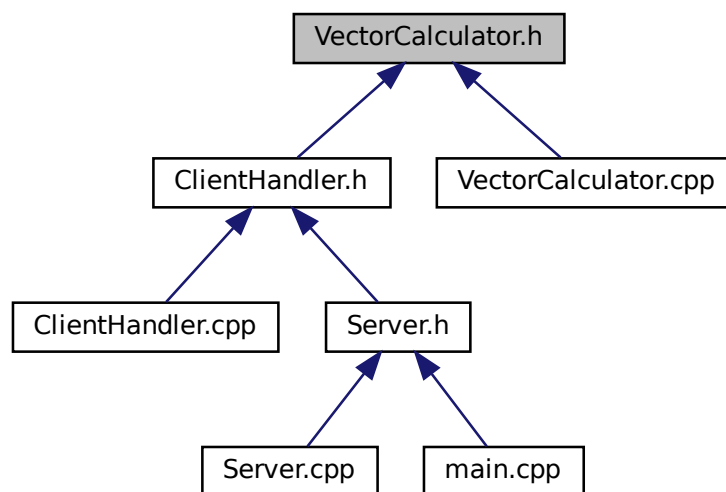
```
#include <vector>
```

```
#include <cstdint>
```

Граф включаемых заголовочных файлов для VectorCalculator.h:



Граф файлов, в которые включается этот файл:



Классы

- class [VectorCalculator](#)

Класс для вычисления произведения элементов вектора

4.30 VectorCalculator.h

[См. документацию.](#)

```
00001 #ifndef VECTORCALCULATOR_H
00002 #define VECTORCALCULATOR_H
00003
00004 #include <vector>
00005 #include <stdint>
00006
00014 class VectorCalculator {
00015 public:
00022     static uint64_t computeProduct(const std::vector<uint64_t>& vector);
00023 };
00024
00025 #endif
```

Предметный указатель

- ~Logger
 - Logger, [23](#)
- ~Server
 - Server, [27](#)
- authenticate
 - Authenticator, [6](#)
- authenticateClient
 - ClientHandler, [12](#)
- Authenticator, [5](#)
 - authenticate, [6](#)
 - computeSHA1, [6](#)
 - generateSalt, [6](#)
- Authenticator.cpp, [35](#)
- Authenticator.h, [36](#)
- be64toh
 - ClientHandler.cpp, [40](#)
- cleanup
 - Server, [28](#)
- clientAddr
 - ClientHandler, [17](#)
- ClientBase, [7](#)
 - ClientBase, [8](#)
 - clients, [10](#)
 - filename, [10](#)
 - getPassword, [8](#)
 - load, [9](#)
 - userExists, [9](#)
- clientBase
 - ClientHandler, [17](#)
 - Server, [29](#)
- ClientBase.cpp, [38](#)
- ClientBase.h, [39](#)
- clientBaseFile
 - Server, [29](#)
 - ServerConfig, [30](#)
- ClientHandler, [10](#)
 - authenticateClient, [12](#)
 - clientAddr, [17](#)
 - clientBase, [17](#)
 - ClientHandler, [12](#)
 - clientSocket, [17](#)
 - handle, [12](#)
 - logger, [17](#)
 - receiveData, [13](#)
 - receiveString, [13](#)
 - receiveVectorsAndProcess, [14](#)
 - sendData, [15](#)
 - sendMessage, [16](#)
 - sendSalt, [16](#)
- ClientHandler.cpp, [40](#)
 - be64toh, [40](#)
 - htobe64, [40](#)
- ClientHandler.h, [44](#)
- clients
 - ClientBase, [10](#)
- clientSocket
 - ClientHandler, [17](#)
- CommandLineParser, [18](#)
 - CommandLineParser, [19](#)
 - config, [21](#)
 - getConfig, [19](#)
 - getHelp, [20](#)
 - helpText, [21](#)
 - parse, [20](#)
 - validateRequiredParams, [20](#)
- CommandLineParser.cpp, [46](#)
- CommandLineParser.h, [48](#)
- computeProduct
 - VectorCalculator, [32](#)
- computeSHA1
 - Authenticator, [6](#)
- config
 - CommandLineParser, [21](#)
- ERROR
 - Logger.h, [51](#)
- error
 - Logger, [23](#)
- filename
 - ClientBase, [10](#)
- generateSalt
 - Authenticator, [6](#)
- getConfig
 - CommandLineParser, [19](#)
- getCurrentTime
 - Logger, [24](#)
- getHelp
 - CommandLineParser, [20](#)
- getPassword
 - ClientBase, [8](#)
- handle
 - ClientHandler, [12](#)
- helpText
 - CommandLineParser, [21](#)

- htobe64
 - ClientHandler.cpp, 40
- INFO
 - Logger.h, 51
- info
 - Logger, 24
- initialize
 - Server, 28
- levelToString
 - Logger, 24
- load
 - ClientBase, 9
- log
 - Logger, 25
- logFile
 - Logger, 25
 - Server, 29
 - ServerConfig, 31
- Logger, 21
 - ~Logger, 23
 - error, 23
 - getCurrentTime, 24
 - info, 24
 - levelToString, 24
 - log, 25
 - logFile, 25
 - Logger, 23
 - logMutex, 25
 - warning, 25
- logger
 - ClientHandler, 17
 - Server, 29
- Logger.cpp, 49
- Logger.h, 50
 - ERROR, 51
 - INFO, 51
 - LogLevel, 51
 - WARNING, 51
- LogLevel
 - Logger.h, 51
- logMutex
 - Logger, 25
- main
 - main.cpp, 53
- main.cpp, 52
 - main, 53
 - server, 54
 - signalHandler, 53
- parse
 - CommandLineParser, 20
- port
 - Server, 29
 - ServerConfig, 31
- receiveData
 - ClientHandler, 13
- receiveString
 - ClientHandler, 13
- receiveVectorsAndProcess
 - ClientHandler, 14
- run
 - Server, 28
- running
 - Server, 29
- sendData
 - ClientHandler, 15
- sendMessage
 - ClientHandler, 16
- sendSalt
 - ClientHandler, 16
- Server, 26
 - ~Server, 27
 - cleanup, 28
 - clientBase, 29
 - clientBaseFile, 29
 - initialize, 28
 - logFile, 29
 - logger, 29
 - port, 29
 - run, 28
 - running, 29
 - Server, 27
 - serverSocket, 30
 - stop, 28
- server
 - main.cpp, 54
- Server.cpp, 55
- Server.h, 56
- ServerConfig, 30
 - clientBaseFile, 30
 - logFile, 31
 - port, 31
- serverSocket
 - Server, 30
- signalHandler
 - main.cpp, 53
- stop
 - Server, 28
- userExists
 - ClientBase, 9
- validateRequiredParams
 - CommandLineParser, 20
- VectorCalculator, 31
 - computeProduct, 32
- VectorCalculator.cpp, 58
- VectorCalculator.h, 59
- WARNING
 - Logger.h, 51
- warning
 - Logger, 25